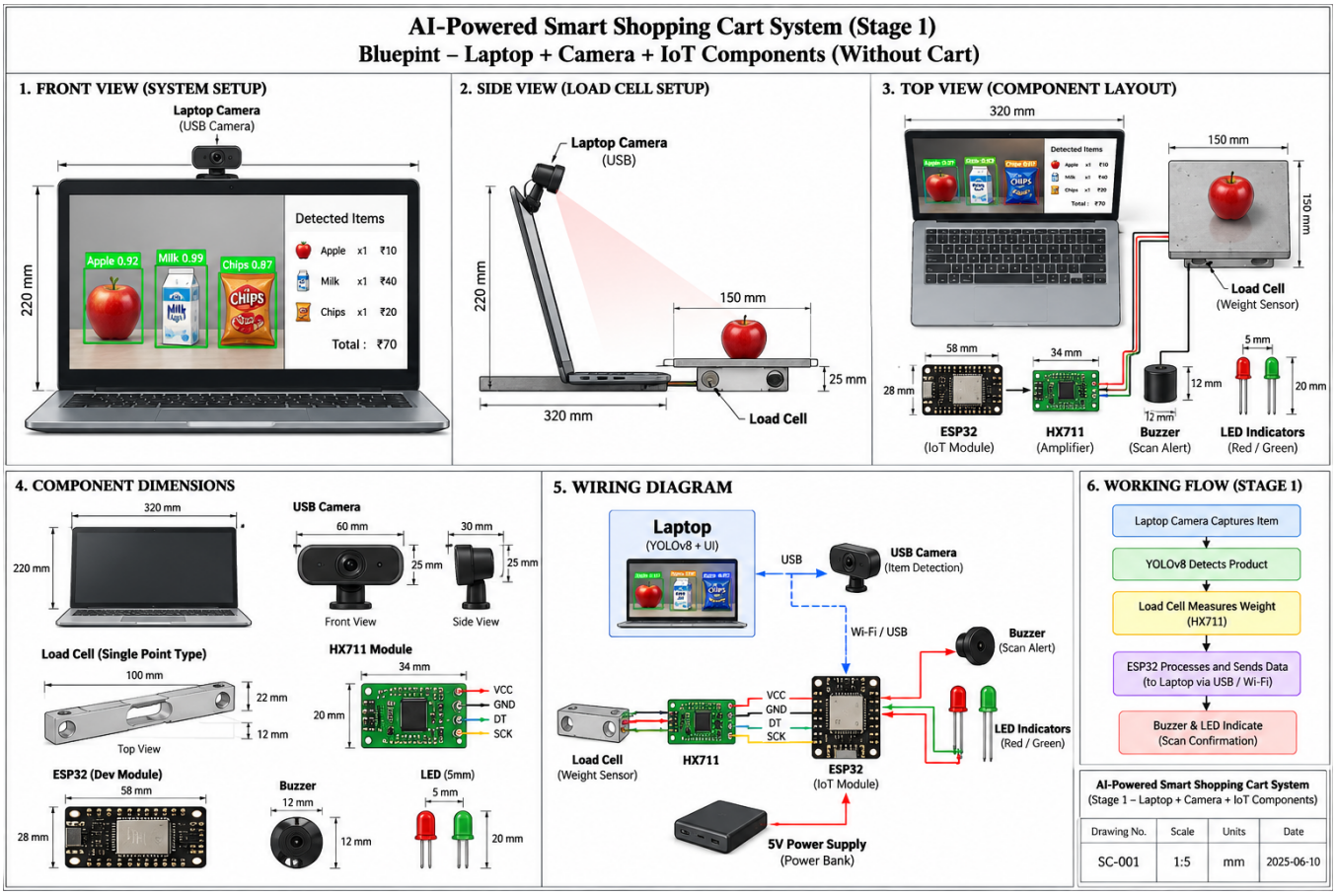
AI-POWERED SMART SHOPPING CART

Stage 1 implementation guide · Laptop + USB camera + IoT components (without a cart)



Goal: build a real-time full-stack application that detects grocery products using a laptop camera, reads weight from a load cell through HX711 + ESP32, looks up product information, updates the cart, and supports a payment/QR workflow.

This guide follows the supplied Stage 1 blueprint. Where the blueprint does not specify an exact software stack or payment provider, recommendations are labelled as suggested choices.

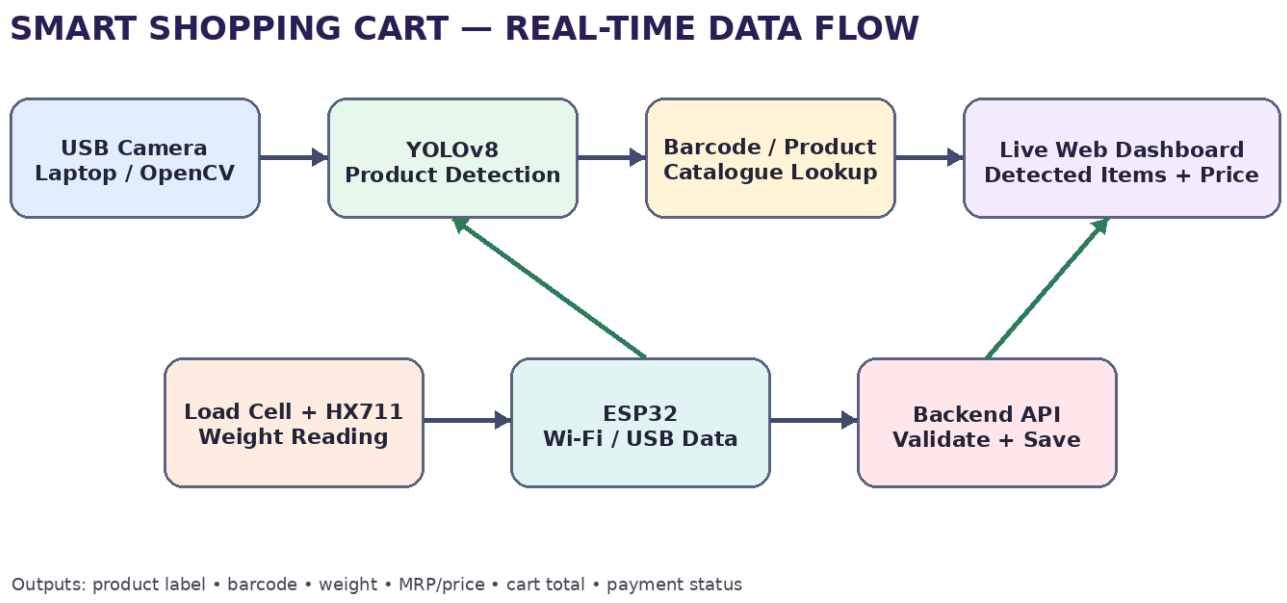
1. Hardware components and their roles

Component	Role in the project	Connection / note
Laptop	Runs the web app, backend, Python vision service and database during development.	Main compute device; start with local development.
USB camera	Captures live video of grocery items for object detection.	USB to laptop; test camera index and lighting.
ESP32 development board	Reads/sends IoT data and can control buzzer/LED indicators.	USB serial or Wi-Fi to laptop/backend.
Single-point load cell	Measures the force/weight of an item placed on the weighing platform.	Connects to HX711; mount firmly and calibrate.
HX711 module	Amplifies and digitizes the small load-cell signal.	Typical interface: DT/DOUT and SCK/CLK to ESP32 GPIO.
Buzzer	Gives an audible alert when a scan or item addition is accepted.	Drive through a suitable transistor/driver if required; check voltage/current.
Red / green LEDs	Show error/ready or scan status at a glance.	Use current-limiting resistors and appropriate GPIO wiring.
5 V power bank	Provides portable power for supported components.	Check ESP32 board input and regulator requirements.

Important electrical note

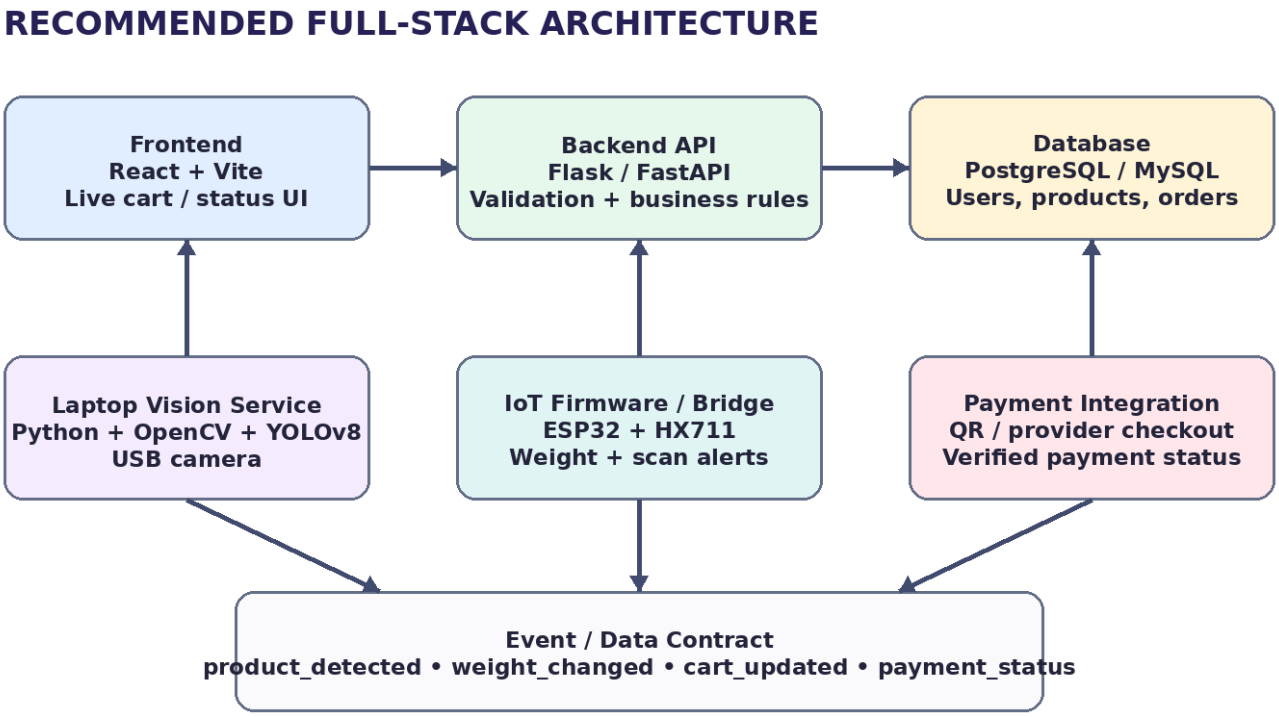
Do not connect a load cell directly to ESP32 analog input: its signal is very small and requires the HX711. Confirm your exact ESP32 board pinout and module voltage before wiring. Use resistors for LEDs and do not power a buzzer directly from a GPIO unless its specifications allow it.

2. System data flow



Suggested event flow: camera frame → object detection → product identity/barcode lookup → product record → cart update. In parallel, load-cell readings travel through HX711 and ESP32 to the application. The backend validates prices and stores orders.

3. Recommended full-stack architecture



Suggested stack

Layer	Recommended option	Why
Frontend	React + Vite; CSS/Tailwind; WebSocket or polling	Live product list, weight, cart and payment status.
Backend	Python Flask or FastAPI	REST endpoints, validation and connection to vision/IoT services.
Vision	Python, OpenCV, YOLOv8	Read camera frames and detect supported grocery classes.
IoT firmware	ESP32 Arduino framework or ESP-IDF	Read HX711; control LEDs/buzzer; send structured events.
Database	PostgreSQL or MySQL	Persist products, barcodes, users, carts, orders and payments.
Real-time bridge	USB serial for first prototype; Wi-Fi HTTP/WebSocket later	Start simple and reliable before adding network complexity.

4. Real-time application workflow

A. Start the services — Run the database, backend API, frontend development server, camera/YOLO process and ESP32 serial reader.

B. Detect a product — OpenCV reads frames; YOLOv8 returns class labels, confidence scores and bounding boxes. Require a stable detection across several frames to avoid duplicate additions.

C. Identify the exact SKU — If exact product identity matters, scan a barcode or use a curated mapping. YOLO class labels such as “milk” or “chips” do not uniquely identify brand, package size or MRP.

D. Read weight — HX711 samples the load cell; ESP32 filters noise and sends a calibrated weight in grams. Tare the platform and calibrate using known weights.

E. Match product and weight — Use the barcode/SKU plus weight rules to verify the product. Do not infer package weight solely from the object-detection model.

F. Update cart — Backend validates the product, quantity and current catalogue price, then sends a cart_updated event to the UI.

G. Signal feedback — ESP32 activates a buzzer or LED only after the application acknowledges the scan/add event; define red/green meanings consistently.

H. Checkout — Calculate the bill on the backend. Generate a payment request through a chosen provider; verify payment server-side before marking the order paid.

5. Suggested API endpoints

Method + endpoint	Purpose
POST /api/auth/login	Authenticate user and issue a session/token.
GET /api/products/barcode/{code}	Return product name, barcode, MRP, net weight and image.
GET /api/products	List searchable products for manual selection.
POST /api/cart/items	Add a validated product and quantity to the active cart.
PATCH /api/cart/items/{id}	Update quantity or item selection.
DELETE /api/cart/items/{id}	Remove an item from the cart.
POST /api/iot/weight	Receive ESP32 weight readings with device ID and timestamp.
POST /api/orders	Create an order and compute authoritative totals.
POST /api/payments/create	Create a payment session/QR request with provider.
GET /api/orders/{id}/status	Read order and payment status.

6. Suggested database structure

Table	Important fields
users	id, username, email, password_hash, created_at
products	id, barcode, name, brand, MRP, net_weight_g, category, image_url, active
devices	id, device_name, esp32_id, status, last_seen_at
weight_readings	id, device_id, weight_g, stable, recorded_at
carts	id, user_id, status, updated_at
cart_items	id, cart_id, product_id, quantity, unit_price, line_total
orders	id, user_id, status, subtotal, discount, tax, total, created_at
payments	id, order_id, provider, provider_reference, amount, status, paid_at

7. Step-by-step build plan

Stage 1 — prove each component
Test USB camera; run YOLO on a sample image/video; read HX711 values in serial monitor; toggle LEDs and buzzer from ESP32.

Stage 2 — device bridge
Send stable weight readings from ESP32 to laptop over USB serial. Use one documented message format, e.g. JSON lines: {"weight_g": 248.5, "stable": true}.

Stage 3 — product catalogue
Create product records with verified barcode, product name, MRP, weight and image. Build barcode lookup and manual fallback.

Stage 4 — backend + database
Create authentication, product, cart and order endpoints. Keep all price calculations on the server.

Stage 5 — live frontend
Build login, dashboard, detected-item list, weight status, cart total and error messages. Add WebSocket updates or short polling.

Stage 6 — checkout
Integrate a test-mode payment provider/QR workflow and verify success/failure callbacks. Never store payment secrets in frontend code.

Stage 7 — end-to-end testing
Test duplicate detection, unknown barcode, noisy weight, unplugged ESP32, network loss, invalid item and failed payment.

8. Real-time reliability and safety

Debounce detections: Add an item only after stable detection or a user confirmation; otherwise one physical item may be added repeatedly.

Calibrate the load cell: Tare when empty and calibrate using known weights. Smooth readings and define a stability threshold.

Separate detection from identification: YOLO detects visual classes; barcode/catalogue lookup provides exact product identity and price.

Treat prices as trusted data: Never trust a price supplied by the browser or an object-detection label. Retrieve prices from the backend catalogue.

Protect user accounts: Hash passwords, validate inputs, use HTTPS outside local development, and restrict device/API credentials.

Verify payment: A displayed QR or uploaded screenshot is not proof of payment. Confirm the provider's server-side transaction status.

Handle disconnects: Show camera/ESP32 offline states, timestamps and retry controls; do not silently keep stale weight values.

9. Acceptance checklist

- ■ Camera feed starts and detects the classes the model was trained for.
- ■ Barcode lookup returns the correct product, MRP and weight from the database.
- ■ Load-cell readings are calibrated and stable enough for the demo.
- ■ ESP32 sends readings with timestamp/device ID and can operate LED/buzzer alerts.
- ■ Dashboard updates product/weight/cart state without refreshing the page.
- ■ Cart totals are calculated and validated on the backend.
- ■ Checkout can show pending/success/failure states based on verified payment response.
- ■ The app handles unknown products, sensor disconnects and network failures clearly.

Final recommendation

Build Stage 1 as a laptop-based prototype first: USB camera + YOLOv8 on the laptop, load cell + HX711 on ESP32, USB serial as the first IoT bridge, React UI, Python API and a local PostgreSQL/MySQL database. Once the data flow is reliable, add Wi-Fi transport and a real payment provider. This is easier to debug than introducing all components at once.

The wiring and component dimensions in the supplied blueprint are illustrative. Verify exact board pinouts, operating voltages, load-cell ratings and power requirements against the datasheets for your purchased components before connecting hardware.